

Lecture 2 Operating Systems Lecture Notes

1 Operating system foundations

Definition

An operating system, or OS, is system software that manages computer hardware and software resources and provides services to application programs and users. It coordinates processor time, memory, storage and devices. It also supplies an environment in which applications can work.

Why an OS is needed

When a student opens a word processor, saves a file and prints a page, the application needs underlying services. The OS manages the application process, supplies memory, controls file access and coordinates the printer. The application performs the user's task while the OS supports its execution.

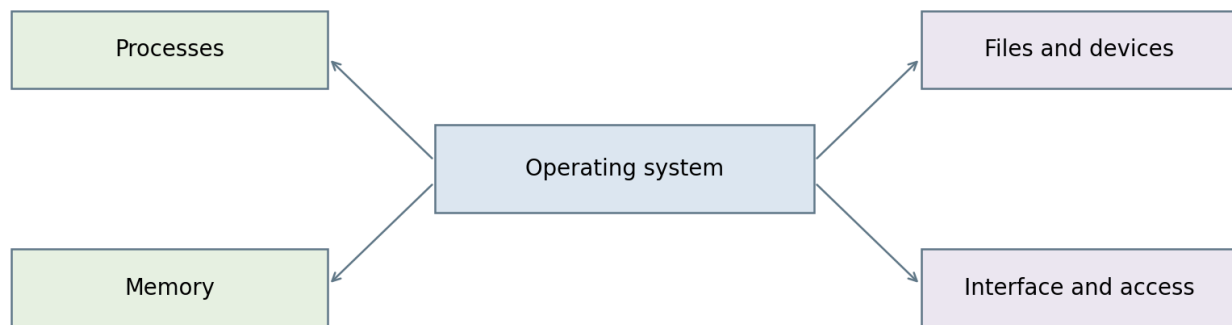


Figure 1 Main responsibilities of an operating system

Important functions

- **Process management:** starts, schedules and ends programs in execution.
- **Memory management:** allocates and tracks RAM used by processes.
- **File management:** organises stored files and controls their access.
- **Device management:** coordinates input and output with suitable drivers.
- **User interface:** supplies graphical interaction or a command environment.
- **Security and permissions:** controls access to supported resources.
- **Networking:** supports communication through available network hardware and software.

Key terms

A process is a running instance of a program. A kernel is the central part of an OS responsible for core operations. A device driver helps the OS communicate with particular hardware. These components work together with other services and tools.

Important distinction

Windows is an operating system; Microsoft Word is an application. Android is an operating system; a messaging app is an application. A browser is also application software. An OS is more than the visible desktop or home screen.

1 Desktop mobile and operating environments

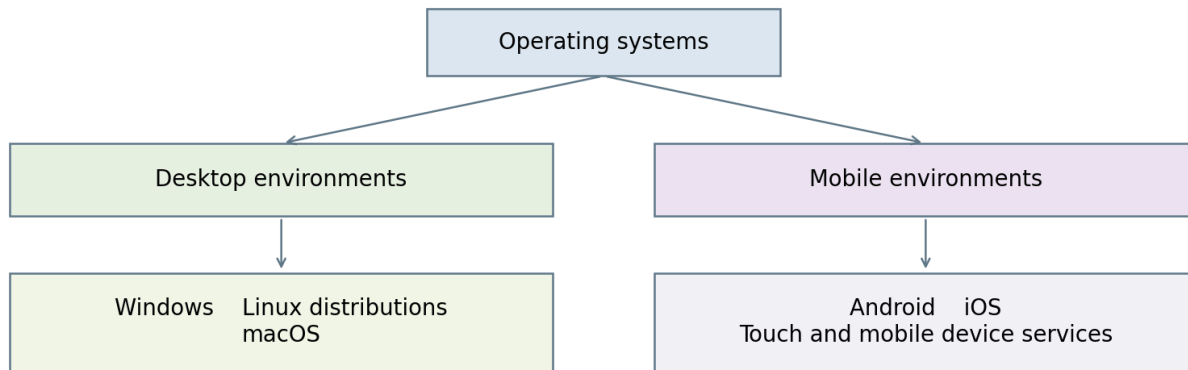


Figure 2 Common desktop and mobile operating environments Desktop operating systems

Desktop operating systems are commonly used on laptops and desktop computers. They support keyboard and mouse interaction, window-based multitasking and general-purpose applications. Windows, macOS and many Linux distributions are examples. Some support touch input as well.

Mobile operating systems

Mobile operating systems are designed for devices such as smartphones. They emphasise touch interaction, battery management, sensors, cameras and wireless connectivity. Android and iOS are examples. Their background activity and resource policies reflect the needs of mobile devices.

GUI and CLI

A graphical user interface, or GUI, uses windows, icons, menus and buttons. A command line interface, or CLI, accepts typed commands. Windows, Linux and macOS can offer both kinds of interaction. Linux is not limited to a command line; desktop distributions provide graphical environments.

Booting

Booting is the process of starting the computer and loading its operating environment. In a simplified sequence, firmware starts, the boot process loads the OS, services initialise and a user session becomes available. A restart repeats the startup process without necessarily switching off power completely.



Figure 3 Simplified operating system startup sequence

Local and remote execution

A local OS runs on the user's own device. A remote desktop OS runs on another computer and sends its screen to the user through a connection. The local device still needs software to display that session and handle input. A VM can run locally or on a remote server.

2 Microsoft Windows

Introduction

Windows is a family of operating systems developed by Microsoft. Desktop Windows is commonly used on personal computers for education, office work, business applications and other tasks. Microsoft also provides server editions for server workloads. These notes concentrate on the desktop environment.

Main features

- Graphical desktop: applications appear in windows that can be moved and resized.
- Start interface and taskbar: provide access to applications and information about running work.
- File Explorer: helps users browse files, folders and storage locations.
- Multitasking: supports several applications during the same period.
- Accounts and permissions: separate user identities and control supported access.
- Hardware support: drivers help connect compatible printers, displays and other devices.
- Settings and management tools: allow configuration and inspection of system resources.
- Networking: supports connections to local networks and online services.

Familiar interface terms

Component	Purpose
Desktop	Main graphical workspace
Taskbar	Access to pinned or running apps and system status
File Explorer	Browsing and managing files and folders
Settings	Changing supported system options
Task Manager	Inspecting applications, processes and resource use

Classroom example

A student opens a document, a spreadsheet and a browser. Windows manages their processes and memory. File Explorer helps locate the assignment, while the applications provide editing and browsing. A print request passes through the operating system and appropriate printer support.

Typical uses

- Document preparation and office productivity.
- Teaching and computer-lab work.
- Business applications and many specialised desktop programs.
- Entertainment and supported games.

2 Windows advantages and limitations

Advantages

- Familiar graphical interface: many users can carry out routine tasks visually.
- Wide software ecosystem: many desktop applications provide Windows versions.
- Broad device ecosystem: numerous compatible computers and peripherals are available.
- Multitasking and file tools: supports common office and educational workflows.
- Management facilities: tools help monitor resources and configure the system.

Limitations

- Licensing: Windows generally requires an appropriate commercial licence, including where it is supplied with a computer.
- Resource requirements: supported versions need suitable hardware and storage.
- Compatibility is not universal: some devices or older applications need different drivers or arrangements.
- Maintenance: updates, configuration and application upkeep remain necessary.
- Security depends on use: unsafe software and poor account settings can create risks.

3 Linux

Introduction

Linux is an open-source kernel used in many operating systems. In everyday introductory usage, “Linux” also refers to complete operating systems built around that kernel. A Linux distribution combines the kernel with software, libraries, management tools and, in desktop editions, a graphical environment.

Kernel and distribution

The kernel handles core interactions with hardware and resources. A distribution provides a usable package around it. Ubuntu, Debian and Fedora are examples of Linux distributions. They differ in packaging, defaults, support arrangements and release policies. Linux is therefore not one identical desktop interface.

Meaning of open source

Open-source software makes source code available under a licence that permits specified uses, study, modification and redistribution. Open source does not automatically mean that every associated service is free of cost. Organisations may pay for support, training or maintenance.

Main features

- Open-source foundation: source can be studied and adapted under its licence.
- Graphical and command interfaces: desktop environments support visual interaction while terminals support commands.
- Multiuser operation: accounts and permissions support separate users and roles.
- Multitasking: coordinates multiple processes.
- Package management: distributions provide tools to install and update software from configured sources.
- Customisation: users and organisations can select many components and settings.
- Networking and server capabilities: widely used for remote services and infrastructure.

Typical uses

- Desktop education and programming environments.
- Web, database and other servers.
- Cloud infrastructure and virtual-machine guests.
- Embedded systems and specialised appliances.

Simple example

A student can use an Ubuntu desktop to write an assignment, open a browser and practise programming. A server may run a Linux distribution without a full desktop interface to provide a website. Both environments use Linux, but their user-facing tools and purposes differ.

3 Linux advantages and limitations

Advantages

- Flexibility: many distributions and configurations are available.
- Open-source access: permits study and modification under the applicable licences.
- Cost choices: many distributions can be used without an OS licence fee.
- Useful development tools: suitable environments support programming and administration.
- Server suitability: commonly used for hosting and network services.
- Choice of interfaces: supports both desktop interaction and command-based work.

Limitations

- Learning requirements: unfamiliar administration tasks may need command and configuration knowledge.
- Application availability: some commercial desktop programs do not provide Linux versions.
- Hardware support differences: certain peripherals require extra setup or have limited support.
- Distribution differences: instructions for one distribution may not apply directly to another.
- Support varies: community support and commercial support differ across products.

Common misconceptions

- “Linux has no GUI” is incorrect; many desktop distributions include one.
- “Linux is always free in every sense” is incorrect; support and services can cost money.
- “Linux cannot get malware” is incorrect; security still depends on configuration and maintenance.
- “All Linux distributions are the same” is incorrect; tools and defaults vary.

4 macOS

Introduction

macOS is Apple's operating system for Mac computers. It provides a graphical desktop and core services for applications, files, devices and users. It is designed for the Mac hardware environment and is distinct from iOS, which is used on iPhone.

Main features

- Graphical desktop: supports windows and visual interaction.
- Finder: helps users browse and manage files and folders.
- Dock: provides convenient access to applications and related items.
- Menu bar: presents menus and system information.
- Multitasking: allows compatible applications to work during the same period.
- System settings: manages supported preferences, accounts and devices.
- Unix foundation: supports a terminal and many command-based tools.
- Apple service integration: can work with supported Apple devices and account services.

Interface terms

Component	Purpose
Finder	Browse files and folders
Dock	Access apps and related items
Menu bar	Application menus and system status
Terminal	Use a command interface
System Settings	Configure supported system options

Uses

- Office work, teaching and everyday personal computing.
- Creative tasks with suitable design, audio or video applications.
- Software development, including supported Apple-platform development tools.
- Working with other supported Apple devices and services.

Advantages and limitations

macOS provides integration between Apple hardware and software, a consistent desktop environment and useful development tools. Its limitations include being designed for the Mac platform, the cost of suitable hardware and differences in application availability. A Windows application does not automatically run on macOS simply because both are desktop operating systems.

5 Android

Introduction

Android is a mobile operating platform based on the Linux kernel. Its open-source foundation is developed through the Android Open Source Project, or AOSP. Device manufacturers can adapt the platform, and commercial devices may include additional proprietary components and services.

Main features

- Touch interface: supports taps, swipes and other gestures.
- Application environment: supplies services that mobile apps use.
- Notifications: informs users about messages and other events.
- Device integration: supports cameras, sensors and wireless hardware through the platform and device support.
- Permissions: controls supported access to resources such as the camera and location.
- Power management: manages activity with battery and mobile use in mind.
- Manufacturer customisation: interfaces and bundled features can differ among devices.

A simple layered view

An Android application uses platform interfaces and services. Those services work with lower-level components and device support, while the kernel manages core resources and hardware interaction. AOSP provides the source platform; it does not mean that every app or commercial phone component is open source.

Typical uses

- Communication and internet access on smartphones.
- Mobile education and productivity applications.
- Navigation and location-based services with appropriate permissions.
- Photos, media and supported entertainment apps.

Advantages

- Available across devices from many manufacturers.
- Adaptable platform and a large application ecosystem.
- Supports many mobile hardware functions.
- Users can choose devices with different designs and capabilities.

Limitations

- Features and update arrangements vary by manufacturer and device.
- Applications may behave differently across hardware and OS versions.
- Untrusted installation sources or excessive permissions can increase risk.
- Mobile resource and battery limits affect what apps can do.

6 iOS

Introduction

iOS is Apple's operating system for iPhone. It manages the phone's resources and provides the environment for compatible applications. Apple's iPad uses the related but separately named iPadOS. Students should not treat macOS, iOS and iPadOS as interchangeable names.

Main features

- Touch interaction: uses gestures and on-screen controls.
- Application environment: provides services for compatible iPhone apps.
- Notifications and settings: helps manage information and device preferences.
- Permissions and app isolation: controls access to supported data and hardware functions.
- Device integration: works with the iPhone camera, sensors and connectivity.
- Power management: handles activity and resources within mobile constraints.
- Apple ecosystem features: supports connections with compatible Apple services and devices.

Uses

- Calls, messaging and online communication.
- Mobile education and productivity.
- Photography, navigation and media use.
- Working with supported account and device services.

Advantages

- Close integration of Apple hardware and software.
- A consistent platform design across supported iPhone models.
- Platform controls for applications and access to resources.
- Convenient integration with compatible Apple devices and services.

Limitations

- Designed for Apple's supported iPhone hardware.
- Less freedom to alter core platform components than many open-source environments.
- Application distribution and system behaviour depend on Apple's rules and supported features.
- Hardware cost, device compatibility and support periods affect use.

7 Comparing the desktop operating systems

Basis	Windows	Linux distributions	macOS
Development	Microsoft	Communities and organisations around Linux	Apple
Typical hardware	Compatible PCs	Supported PCs, servers and other devices	Mac computers
Licensing approach	Generally commercial proprietary OS	Open-source foundation; costs vary	Apple proprietary OS with open-source components
Graphical file tools	File Explorer	Depends on the desktop environment	Finder
Command interface	Supported shells and terminals	Common administration tools	Terminal and supported shells
Typical strengths	Broad desktop software ecosystem	Flexibility and server/development use	Integration with Mac hardware
Possible limitation	Licensing and resource requirements	Some apps and devices need alternatives	Mac hardware and app compatibility restrictions

7 Android iOS and mobile computing compared

Basis	Android	iOS
Platform	Linux-based mobile platform with AOSP foundation	Apple mobile OS for iPhone
Typical devices	Devices from many manufacturers	Supported Apple iPhone models
Interface	Touch interface; manufacturer variations	Touch interface designed by Apple
Customisation	Varies by device and implementation	More centrally controlled platform design
Updates	Depend on device and supplier arrangements	Provided for supported Apple devices
Applications	Android-compatible apps	iOS-compatible apps
Shared functions	App management, sensors, permissions, power	App management, sensors, permissions, power

Mobile versus desktop requirements

A mobile OS must pay particular attention to battery use, small screens, wireless networks and sensors. A desktop OS commonly prioritises keyboard/mouse work, larger windows and many desktop applications. The distinction is a design emphasis, not an absolute rule: modern devices and software can offer overlapping capabilities.

Example of permissions

A camera app needs access to the camera. A navigation app may need location access. A permission system gives supported ways to control such requests. Denying an unnecessary request can reduce exposure, but denying a permission needed for a feature can prevent that feature from working.

Background work

A phone may pause or restrict an app when it is not actively being used to save resources. Desktop and mobile systems both manage multitasking, but their policies differ. Therefore, a mobile application cannot assume unlimited background execution simply because a desktop application can run for a long time.

Common misconceptions

- Android is not the same as a standard desktop Linux distribution, although it uses the Linux kernel.
- iOS is not the Mac operating system; macOS is.
- An Android app and an iOS app do not automatically use the same installable package.
- A large app ecosystem does not guarantee that every app is safe or suitable.

8 Basics of cloud based operating environments

Meaning of cloud computing

Cloud computing provides computing services through networks using remotely managed infrastructure. Services can include storage, applications, processing and virtual computers. The hardware still exists in data centres or other locations. The word cloud does not mean that computation occurs without physical machines.

Meaning of cloud based OS

The phrase “cloud-based OS” is used broadly. It can describe a local operating environment designed around web applications and cloud storage, or an operating desktop hosted remotely and accessed over a network. These are different arrangements and should be explained separately.

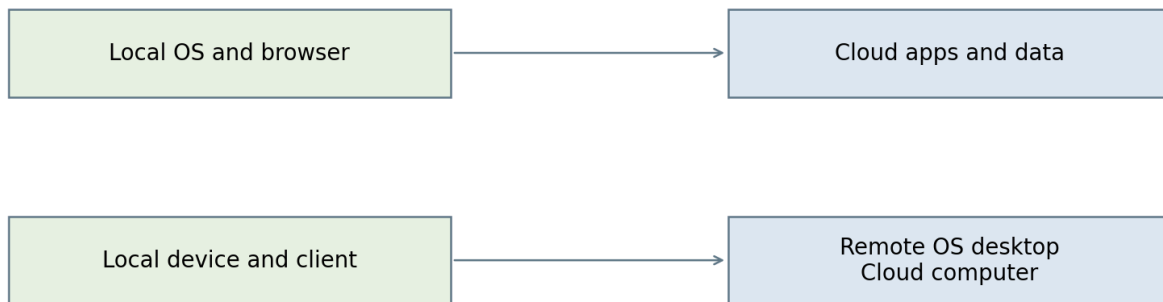


Figure 5 Cloud focused local systems and remotely hosted desktops are different models

Model 1 Local OS with strong cloud integration

The operating system runs on the local device. A browser or app connects to remote services for some data and processing. ChromeOS is a common cloud-focused example. It still runs locally and has local operating-system components. Some supported tasks can work offline when suitable apps and setup are available.

Model 2 Remotely hosted operating desktop

The operating desktop runs on a remote computer, commonly a virtual machine. The user connects through a client or browser. Input travels to the remote system, and the screen output returns. A Cloud PC service such as Windows 365 illustrates this model. The client device still uses its own operating environment.

A cloud application is not an OS

Google Docs and Google Drive are applications or services, not operating systems. Opening them in a browser does not replace the OS managing the local computer. A cloud-focused workflow can combine a local OS with remote apps and data.

8 Cloud operation advantages and limitations

How a remote desktop session works

1. A user opens a supported client or browser on a local device.
2. The user signs in and connects to an authorised remote desktop.
3. The remote OS and its applications run on the remote infrastructure.
4. The connection carries input actions and display information.
5. Files are stored according to the service's storage configuration.

Advantages

- Access from different supported devices: users can connect to their environment where access is permitted.
- Central management: an organisation can manage desktop environments from a common service.
- Remote resources: a service can supply computing capacity beyond the local device's role.
- Continuity of work: files and apps may be available from the same remote session.
- Collaboration: cloud services can make shared work and information access easier.

Limitations

- Network dependence: a remote desktop needs suitable connectivity; poor connections cause delay or interruption.
- Service availability: provider or network faults can affect access.
- Recurring cost: hosted resources and subscriptions can require ongoing payments.
- Privacy and access: organisations must manage where data are stored and who can access them.
- Peripheral support: printers, cameras or other local devices may require extra arrangements.
- Offline restrictions: functions depending on remote execution cannot normally continue when disconnected.

Cloud focused does not mean always online

A locally running cloud-focused OS may support some offline apps or downloaded files. A fully remote desktop is different because its processing happens elsewhere. Students should describe the arrangement rather than apply the same offline claim to every cloud system.

8 Local systems cloud applications and Cloud PCs

Basis	Local OS and apps	Cloud application	Remote Cloud PC
Where OS runs	On the user's device	Local device still has an OS	Desktop OS runs remotely; client has its own OS
Main service	Local desktop computing	A specific online task	A remote desktop environment
Example	A local desktop running Writer	Google Docs in a browser	Windows 365 Cloud PC
Data location	Usually local, with optional remote storage	According to application setup	According to remote service setup
Offline work	Possible for supported local apps	Depends on offline facilities	Remote execution normally requires a connection

Important distinctions

- Cloud storage stores data remotely; it is not the same as a complete remote desktop.
- Remote access to a home PC is not automatically a cloud service; the location and management arrangement matter.
- A cloud service may use VMs internally even when the user never sees them.
- A local computer can also run a VM without any cloud service.

9 Virtual machines

Definition

A virtual machine, or VM, is a software-defined computer environment that provides virtual hardware on which a guest operating system can run. It behaves like a separate computer for many purposes while using resources from an underlying physical machine. Virtualisation is the technology used to create and manage such environments.

Physical and virtual computers

A physical computer has real processors, RAM, drives and network devices. A VM receives virtual versions of these resources. The guest OS uses the virtual hardware, and virtualisation software maps its operations onto available physical resources.

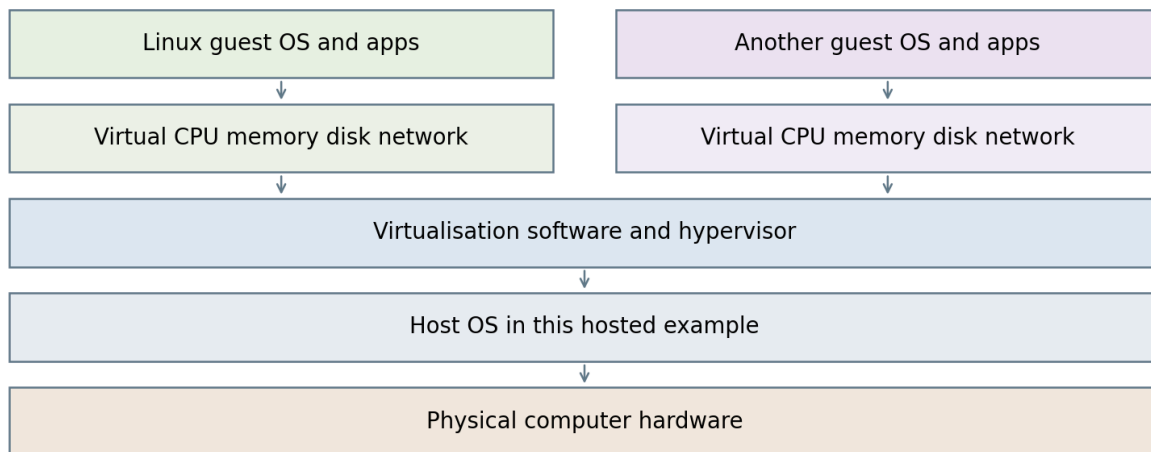


Figure 6 A hosted virtualisation example with separate guest systems sharing physical hardware

Simple classroom example

A Windows computer can run a supported virtualisation application and an Ubuntu VM. Windows belongs to the host environment, Ubuntu is the guest OS, and the virtualisation system supplies virtual hardware. Students can practise Linux while keeping their normal host environment available.

The diagram is a simplified hosted arrangement. Some hypervisors operate directly on server hardware, so an ordinary desktop host OS is not always a separate layer.

9 Hypervisors and creating a VM

Type 1 hypervisor

A Type 1, or bare-metal, hypervisor runs directly on the hardware in the usual introductory classification. It is commonly used in server virtualisation. VMware ESXi is a familiar example. Real implementations can involve management operating environments and additional services.

Type 2 hypervisor

A Type 2, or hosted, hypervisor operates through a host operating system in the usual introductory classification. It is convenient for learning and desktop experiments. Oracle VirtualBox is a familiar example. Students should focus on the difference between direct hardware deployment and a hosted environment.

Basis	Type 1	Type 2
Underlying environment	Direct hardware deployment in the basic model	Existing host OS in the basic model
Common setting	Server and data-centre virtualisation	Desktop learning and testing
Example	VMware ESXi	Oracle VirtualBox
Main learning point	No ordinary desktop host layer required	Runs within a host operating environment

General steps to create a VM

1. Check that the machine and virtualisation tool support the intended guest OS and processor architecture.
2. Obtain an appropriate OS installation image and any required licence.
3. Create a VM and choose the guest type.
4. Allocate a suitable amount of virtual memory, processor resources and disk space.
5. Attach the installation image and start the VM.
6. Install the guest OS onto its virtual disk.
7. Configure supported integration and network settings, then use the guest.

Resource allocation example

On a host with 16 GB RAM, a teacher might assign 4 GB to a small guest as an illustration. That memory allocation must still leave enough resources for the host and its applications. Requirements depend on the guest and workload. Virtual resources do not create unlimited physical capacity.

Architecture compatibility

A VM must be compatible with the virtualisation platform and available processor support. Running an OS built for a different architecture may require emulation, which is a related but distinct technique and may have additional overhead. Not every OS can run on every host.

9 VM uses advantages and limitations

Uses

- Learning another OS without dedicating a separate physical computer.
- Testing applications in a separate configured environment.
- Running multiple server workloads on one physical system.
- Maintaining environments for supported older applications.
- Providing cloud virtual computers and remotely hosted desktops.

Advantages

- Several environments: one machine can support multiple guest systems.
- Convenient testing: experiments can use a separate guest configuration.
- Resource sharing: workloads can share available physical hardware.
- Manageable state: supported tools can save configurations and snapshot states.
- Deployment flexibility: virtual disk and configuration data can assist moving supported workloads.

Limitations

- Resource demand: the host and guests share processor, memory and storage.
- Possible overhead: some work can be slower than direct native execution.
- Compatibility limits: CPU architecture, devices, drivers and licences matter.
- Hardware access: specialised devices and graphics can require additional support.
- Host dependence: failure of the underlying machine can affect several VMs.
- Isolation is not perfect: integration features or vulnerabilities can allow unwanted interactions.

Snapshots and backups

A snapshot records a VM state according to the tool's capabilities. It can help return to an earlier experimental state. A snapshot is not automatically an independent backup; it may depend on the original virtual disk and the same storage system. Important data need a recoverable backup arrangement.

Shared resources

Shared folders, clipboard integration and network connections make host-guest interaction convenient. They also reduce separation for those resources. A guest can contain unsafe files, so a VM should not be described as an environment where every action is harmless.

9 Virtual machines dual boot and cloud compared

Basis	VM	Dual boot
Meaning	Guest OS runs through virtualised hardware	Device has alternative OS installations selected at boot
At the same time	Host and guest can run during the same period	Normally one selected OS runs at a time
Switching	Use the running virtual environment	Usually restart and choose another OS
Hardware access	Through supported virtualisation mechanisms	Selected OS uses physical hardware directly
Resources	Shared with the host environment	Available to the currently booted OS